

Dispositivos Conectados Conectividade Wi-Fi

2025/2026

Authors: Paulo C. Bartolomeu

1 Objetivos

- Familiarização com os modos de operação do Wi-Fi no ESP32-C6;
- Utilização dos modos *Station* e *SoftAP*;
- Implementação e teste de servidor HTTPS básico com conectividade Wi-Fi.

2 Introdução

O Wi-Fi é uma marca registada de tecnologia de comunicação sem fios detida pela Wi-Fi Alliance (WFA), a entidade responsável pela supervisão do processo de certificação Wi-Fi. Trata-se de uma família de protocolos de rede sem fios baseada nos padrões IEEE 802.11. Os produtos que passam por testes rigorosos recebem o selo Wi-Fi CERTIFIED™, comprovando que cumprem os requisitos definidos pela indústria em termos de interoperabilidade, segurança e vários protocolos específicos de aplicação.

Em comparação com outras tecnologias de comunicação sem fios, o Wi-Fi oferece maior cobertura, melhor capacidade de penetração através de paredes e débitos de transmissão mais elevados.

O elemento fundamental de uma rede IEEE 802.11 é o Basic Service Set (BSS). Este pode apresentar duas categorias: Independent BSS e Infrastructure BSS. Num infrastructure BSS, as estações (STAs) têm de se associar a um Ponto de Acesso (AP) para obter serviços de rede. Os AP funcionam como o centro de controlo do conjunto, sendo esta a arquitetura de rede mais comum. Cada STA deve passar pelos processos de associação e autorização antes de entrar num determinado BSS.

Numa perspectiva prática, as arquiteturas de rede Wi-Fi são acompanhadas por identificadores próprios:

- **BSS Identification (BSSID):** Cada BSS possui um endereço físico de identificação, denominado BSSID. Num *Infrastructure BSS*, o BSSID corresponde ao endereço MAC do *Access Point* (AP). Este valor é definido de fábrica, mas pode ser alterado seguindo um formato de nomenclatura específico.
- **Service Set Identification (SSID):** Cada AP dispõe também de um identificador destinado ao utilizador, denominado SSID. Na maioria dos casos, um BSSID está associado a um único SSID. Este identificador é normalmente um texto legível, sendo aquilo que vulgarmente chamamos de nome da rede Wi-Fi.

Dado este enquadramento, é importante compreender como estas mesmas estruturas e identificadores

se aplicam aos dispositivos *embedded* que integram conectividade sem fios. No caso do ESP32-C6, a framework ESP-IDF disponibiliza suporte Wi-Fi completo segundo as normas IEEE 802.11, permitindo que o SoC funcione tanto como *Station* (STA) - ligando-se a um *Access Point* existente - como em SoftAP, criando a sua própria rede. A configuração destas funcionalidades é feita através de APIs específicas que permitem definir o SSID, a segurança, o canal, e gerir eventos de associação, obtenção de endereço IP e perda de conectividade.

A integração do Wi-Fi no ESP32-C6 segue exatamente os princípios descritos anteriormente: ao atuar como STA, o dispositivo passa pelo processo de pesquisa de redes, autenticação e associação a um BSSID; ao atuar como SoftAP, é o próprio ESP que anuncia o SSID, fornece endereços IP por DHCP e gere as estações que se ligam.

3 Trabalho Prático

O trabalho prático desta aula desenvolve-se com base na [API de Referência Wi-Fi do ESP32-C6](#) e na [API de Referência HTTPS Server do ESP32-C6](#), juntamente com o [ESP32-C6 Technical Reference Manual](#).

A documentação da Expressif relativa ao [Driver Wi-Fi](#) é de **leitura obrigatória** para perceber o código dos exemplos.

3.1 Wi-Fi Modo Cliente (STA)

O exemplo do Modo Cliente Wi-Fi demonstra como configurar o ESP32-C6 para funcionar no modo Station (STA), ligando-se a um ponto de acesso Wi-Fi existente, como por exemplo um *Access Point* (AP) criado pelo telemóvel do aluno. O programa inicia o subsistema de rede, tenta ligar-se à rede definida no `menuconfig` e regista eventos para acompanhar o processo de ligação, incluindo tentativas de reconexão e falhas. Quando a ligação é bem-sucedida, o dispositivo obtém um endereço IPv4 via DHCP do AP, e essa informação é apresentada no terminal.

QUESTÃO 1: No VSCode crie um novo projeto a partir do template `wifi -> getting_started -> station`. Usando o comando `menuconfig` defina o SSID e a Password da rede Wi-Fi que tiver configurado no seu telemóvel. O acesso a esta configuração no projecto encontra-se na secção `Example Configuration` do `menuconfig`. Compile o programa e descarregue o *firmware* para o SoC. Deverá observar o seguinte *log* de eventos:

```
...
I (346) main_task: Started on CPU0
I (356) main_task: Calling app_main()
I (376) wifi station: ESP_WIFI_MODE_STA
...
W (546) wifi:(agc)0x600a7128:0xd20729de, min.avgNF:0xce->0xd2(dB), RCalCount:0x72, min.RRssi:0x9de(-98.12)
I (556) wifi:11ax coex: WDEVAX_PTI0(0x000a00a0), WDEVAX_PTI1(0x000052bd).

I (566) wifi:mode : sta (58:8c:81:3b:c0:bc)
...
I (866) wifi:connected with phobar, aid = 1, channel 6, BW20, bssid = 36:14:26:eb:1f:77
I (13586) wifi:security: WPA2-PSK, phy:ax, rssi:-37, cipher(pairwise:0x3, group:0x3), pmf:0,
...
I (14966) esp_netif_handlers: sta ip: 172.20.10.2, mask: 255.255.255.240, gw: 172.20.10.1
I (14966) wifi station: got ip:172.20.10.2
```

QUESTÃO 2: Agora que o ESP32-C6 possui conectividade IPv4, ligue-se com o PC ao mesmo *Access Point* e teste a conectividade com o dispositivo IoT. O teste deverá ser

bem sucedido, com latências reduzidas de poucos milissegundos face ao reduzido número de *hops* na ligação entre os dois *peers*.

QUESTÃO 3: Cause uma disrupção momentânea no AP, por exemplo, desligando-o temporariamente 2/3 segundos e reponha-o em funcionamento. Deverá observar um comportamento consistente com a seguinte listagem:

```
I (88156) wifi:bcn_timeout,ap_probe_send_start
I (90656) wifi:ap_probe_send over, reset wifi status to disassoc
I (90656) wifi:state: run -> init (0xc800)
I (90666) wifi:pm stop, total sleep time: 13463547 us / 23811903 us

I (90666) wifi:<ba-del>idx:0, tid:6
I (90666) wifi:new:<6,0>, old:<6,0>, ap:<255,255>, sta:<6,0>, prof:1, snd_ch_cfg:0x0
I (90676) wifi station: retry to connect to the AP
I (90676) wifi station: connect to the AP fail
I (93086) wifi station: retry to connect to the AP
I (93086) wifi station: connect to the AP fail
...
I (93496) wifi:connected with phobar, aid = 1, channel 6, BW20, bssid = 52:6a:a3:22:c6:39
I (93506) wifi:security: WPA2-PSK, phy:ax, rssi:-34, cipher(pairwise:0x3, group:0x3), pmf:0,
I (93516) wifi:pm start, type: 1, twt_start:0
...
I (94836) esp_netif_handlers: sta ip: 172.20.10.2, mask: 255.255.255.240, gw: 172.20.10.1
I (94836) wifi station: got ip:172.20.10.2
```

Como se pode observar, a ligação deve ser reestabelecida, assim como o IP é novamente obtido.

3.2 Wi-Fi Modo “Soft” *Access Point* (SoftAP)

No exemplo SoftAP vamos configurar o ESP32-C6 para operar como um *Access Point* Wi-Fi. Isto significa que o próprio ESP cria e anuncia uma rede Wi-Fi autónoma, permitindo que outros dispositivos — como *smartphones*, computadores portáteis ou módulos IoT — se possam ligar diretamente a ele, sem necessidade de um *router* externo.

O exemplo inclui um mecanismo de monitorização que imprime mensagens de *log* sempre que uma estação (dispositivo) se liga ao ponto de acesso. Neste âmbito, o ESP regista o endereço MAC do dispositivo, o identificador de associação (AID) e outras informações relevantes, permitindo saber exatamente quando e quem entrou na rede.

QUESTÃO 4: No VSCode crie um novo projeto a partir do template `wifi -> getting_started -> softAP`. Usando o comando `menuconfig` defina o SSID e a Password da rede Wi-Fi na secção `Example Configuration`. Compile o programa e descarregue o *firmware* para o SoC. Deverá observar o seguinte *log* de eventos:

```
I (356) main_task: Started on CPU0
I (366) main_task: Calling app_main()
I (386) wifi softAP: ESP_WIFI_MODE_AP
I (386) pp: pp rom version: 5b8dcfa
...
I (566) wifi:11ax coex: WDEVAX_PTI0(0x000a00a0), WDEVAX_PTI1(0x000052bd).

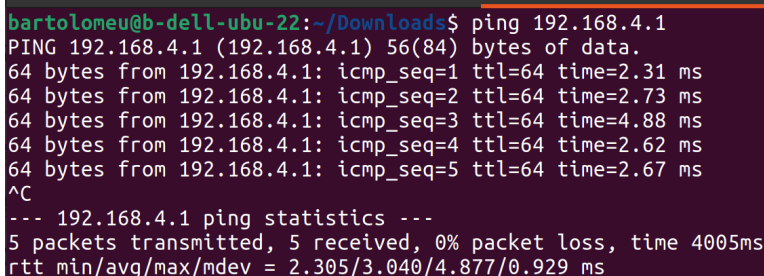
I (576) wifi:mode : softAP (58:8c:81:3b:c0:bd)
I (626) wifi:Total power save buffer number: 16
I (626) wifi:Init max length of beacon: 752/752
I (626) wifi:Init max length of beacon: 752/752
I (626) esp_netif_lwip: DHCP server started on interface WIFI_AP_DEF with IP: 192.168.4.1
I (636) wifi softAP: wifi_init_softap finished. SSID:myssid password:mypassword channel:1
I (646) main_task: Returned from app_main()
```

Nesta fase o ESP opera como um *Access Point* estando pronto para receber ligações e atribuir *leases* DHCP.

QUESTÃO 5: Usando o portátil seleccione a rede Wi-Fi criada pelo ESP32-C6 e ligue-se com a *password* definida. Deverá observar na consola uma mensagem de log com indicação do IP que foi atribuído, tal como indicado na listagem seguinte:

```
I (32756) wifi:station: e2:86:0b:c1:08:a0 join, AID=1, bgn, 20
I (32776) wifi softAP: station e2:86:0b:c1:08:a0 join, AID=1
I (33986) esp_netif_lwip: DHCP server assigned IP to a client, IP is: 192.168.4.2
```

QUESTÃO 6: Utilizando o comando ping teste a conectividade com o Access Point. Deverá observar um resultado semelhante ao da Figura 1.



```
bartolomeu@b-dell-ubu-22:~/Downloads$ ping 192.168.4.1
PING 192.168.4.1 (192.168.4.1) 56(84) bytes of data:
64 bytes from 192.168.4.1: icmp_seq=1 ttl=64 time=2.31 ms
64 bytes from 192.168.4.1: icmp_seq=2 ttl=64 time=2.73 ms
64 bytes from 192.168.4.1: icmp_seq=3 ttl=64 time=4.88 ms
64 bytes from 192.168.4.1: icmp_seq=4 ttl=64 time=2.62 ms
64 bytes from 192.168.4.1: icmp_seq=5 ttl=64 time=2.67 ms
^C
--- 192.168.4.1 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4005ms
rtt min/avg/max/mdev = 2.305/3.040/4.877/0.929 ms
```

Figura 1: Teste de conectividade ao *Access Point*

O modo SoftAP é útil em várias fases do ciclo de vida de um dispositivo IoT. Em primeiro lugar, facilita a configuração inicial, permitindo que o utilizador se ligue diretamente ao dispositivo através do telemóvel para introduzir as credenciais da rede Wi-Fi ou ajustar parâmetros essenciais. Este modo é igualmente vantajoso em cenários onde o dispositivo precisa de funcionar de forma autónoma, sem depender de um router externo, como em instalações temporárias, ambientes industriais isolados ou medições de campo. Além disso, pode ser uma ferramenta muito prática para diagnóstico e manutenção, possibilitando recuperar dispositivos com falhas de ligação, alterar configurações ou realizar testes sem recorrer a cabos ou infraestruturas adicionais. Por fim, o SoftAP permite disponibilizar interfaces de controlo locais, como páginas web de configuração ou dashboards, assegurando comunicação direta sem necessidade de acesso à Internet.

3.3 Servidor Web

Nesta secção vamos abordar o exemplo `https_server/simple` da plataforma ESP-IDF para o SoC ESP32-C6. Este exemplo demonstra como criar um servidor web seguro (HTTPS) a correr diretamente no dispositivo. O programa estabelece uma ligação de rede (por Wi-Fi), carrega um certificado e chave privada embutidos no firmware e inicia um servidor HTTPS que escuta pedidos na porta segura. Em seguida, regista um handler simples para o método GET na rota “/”, respondendo com uma pequena página HTML. O exemplo mostra também como iniciar e parar o servidor automaticamente quando a interface de rede ganha ou perde conectividade, ilustrando o funcionamento básico do stack TLS, do servidor HTTP seguro e da gestão de eventos de rede no ESP32-C6.

Para este exercício, **precisará de configurar o seu telemóvel como *Access Point* de modo a que este atribua um endereço IP ao SoC ESP32-C6 quando este arrancar.**

QUESTÃO 7: No VSCode crie um novo projeto a partir do template `https_server` -> `simple`. Usando o comando `menuconfig` defina o SSID e a Password da rede Wi-Fi criada pelo seu telemóvel na secção `Example Configuration`. Compile o programa e descarregue o `firmware` para o SoC. Deverá observar o seguinte *log* de eventos na consola:

```
...
I (23) boot: ESP-IDF v5.5.1-dirty 2nd stage bootloader
...
I (403) main_task: Started on CPU0
I (403) main_task: Calling app_main()
I (413) example_connect: Start example_connect.
...
I (603) wifi:11ax coex: WDEVAX_PTIO(0x55777555), WDEVAX_PTI1(0x00003377).

I (603) wifi:mode : sta (58:8c:81:3b:c0:bc)
I (613) wifi:enable tsf
I (613) example_connect: Connecting to phobar...
W (623) wifi:Password length matches WPA2 standards, authmode threshold changes from OPEN to WPA2
I (633) example_connect: Waiting for IP(s)
...
W (3513) wifi:<ba-add>idx:0, ifx:0, tid:6, TAHI:0x100104a, TALO:0x9ae3c74a, (ssn:0, win:64, cur_ssn:0),
CONF:0xc0006005
I (4413) example_connect: Got IPv6 event: Interface "example_netif_sta" address:
fe80:0000:0000:0000:5a8c:81ff:fe3b:c0bc, type: ESP_IP6_ADDR_IS_LINK_LOCAL
I (4523) example: Starting server
I (4523) esp_https_server: Starting server
I (4523) esp_https_server: Server listening on port 443
I (4523) example: Registering URI handlers
I (4523) esp_netif_handlers: example_netif_sta ip: 172.20.10.2, mask: 255.255.255.240, gw: 172.20.10.1
I (4533) example_connect: Got IPv4 event: Interface "example_netif_sta" address: 172.20.10.2
I (4543) example_common: Connected to example_netif_sta
I (4543) example_common: - IPv4 address: 172.20.10.2,
I (4553) example_common: - IPv6 address: fe80:0000:0000:0000:5a8c:81ff:fe3b:c0bc, type:
ESP_IP6_ADDR_IS_LINK_LOCAL
I (4563) main_task: Returned from app_main()
```

Neste momento, o servidor básico está ativo, tendo-se conectado ao *Access Point* criado no telemóvel. O AP atribuiu o endereço IP 172.20.10.2 ao ESP32-C6, servindo neste mesmo IP (e porto 80 - *default*) uma página web. Note que, dependendo da configuração do seu AP o endereço IP atribuído poderá (deverá) ser diferente.

QUESTÃO 8: Ligue o seu computador ao Access Point criado pelo telemóvel e aceda ao endereço 172.20.10.2. Como poderá constatar, o *browser* irá apresentar um aviso de segurança, impedindo-o de visualizar imediatamente a página servida pelo ESP32-C6.

Isto acontece porque o certificado utilizado pelo servidor é *self-signed* — ou seja, não foi emitido por uma autoridade de certificação reconhecida (como a Let's Encrypt ou a DigiCert). Assim, o *browser* não tem forma de confirmar a identidade do servidor e considera a ligação potencialmente insegura. A este fator junta-se normalmente outro: o nome indicado no certificado não corresponde ao endereço utilizado para aceder ao dispositivo (por exemplo, estamos a usar `https://172.20.10.2/`, mas o certificado contém outro nome). Em contexto de desenvolvimento e de testes este comportamento é esperado. A solução consiste simplesmente em aceitar a exceção de segurança no navegador ou, alternativamente, gerar um certificado com o nome correto ou instalar a autoridade emissora como confiável no sistema.

Por agora, aceite a exceção de segurança e deverá observar a página web da Figura 2.

QUESTÃO 9: Atualize o programa de modo a que imprima informação sobre a temperatura e humidade instantâneas sempre que seja pedida uma nova página.

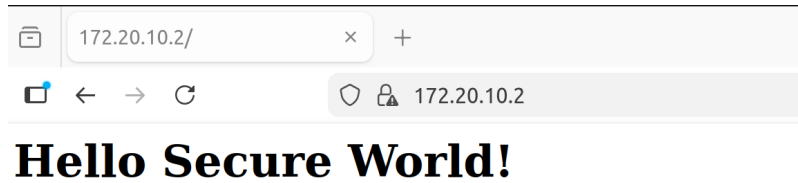


Figura 2: Página obtida do servidor web básico

Este exemplo estabelece um servidor web no ESP32-C6 que opera como um cliente da rede Wi-Fi para obter um endereço IP e ter conectividade. Isso implica que quem quiser aceder ao servidor web tem de “descobrir” o endereço IP atribuído ao ESP32-C6, o que nem sempre é fácil. Uma forma de contornar este problema é estabelecer que o servidor web desempenhe simultaneamente o papel de *Access Point*.

QUESTÃO 10: Altere o programa de modo a que o servidor web seja simultaneamente um softAP que permita a ligação de um cliente Wi-Fi, atribuindo-lhe automaticamente um endereço IP e servindo-lhe uma página web com a temperatura e humidade instantâneas sempre que esta for pedida.

4 Acknowledgements

Originally authored by [Paulo C. Bartolomeu](#)